

The procedure converts a spatially varying loop number density $n(\mathbf{x})$ into a discrete set of loops attached to computational nodes. It has two stages:

1. Assign each node a representative material volume.
2. Use the density within that volume to determine how many loops belong there.

1. Nodal volumes from a restricted Voronoi tessellation

Suppose the crystal contains nodes

$$\{\mathbf{x}_j\}_{j=1}^M.$$

The ordinary Voronoi cell of node j is

$$\mathcal{V}_j = \{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_j\| \leq \|\mathbf{x} - \mathbf{x}_k\|, \quad \forall k\}.$$

This is the region of space closer to node j than to any other node.

Because boundary-node Voronoi cells may extend to infinity, the cells must be restricted to the physical crystal domain Ω :

$$\mathcal{V}_j^\Omega = \mathcal{V}_j \cap \Omega.$$

Here, Ω is approximated by the convex hull of the node cloud,

$$\Omega = \text{conv}\{\mathbf{x}_1, \dots, \mathbf{x}_M\}.$$

The exact nodal volume would therefore be

$$V_j = \text{vol}(\mathcal{V}_j^\Omega).$$

These restricted cells partition the crystal:

$$\Omega = \bigcup_j \mathcal{V}_j^\Omega, \quad \mathcal{V}_j^\Omega \cap \mathcal{V}_k^\Omega \text{ has zero volume for } j \neq k,$$

and consequently

$$\sum_j V_j = V_\Omega.$$

Why use the convex hull?

The enclosing Cartesian bounding box includes space outside the crystal. For a regular hexagonal prism, the bounding rectangle of the hexagonal cross-section has area

$$A_{\text{box}} = 4\sqrt{3}a^2,$$

whereas the regular hexagon has area

$$A_{\text{hex}} = \frac{3\sqrt{3}}{2}a^2.$$

Thus,

$$\frac{V_{\text{box}}}{V_{\text{crystal}}} = \frac{A_{\text{box}}}{A_{\text{hex}}} = \frac{8}{3},$$

for the usual vertex-aligned bounding rectangle. A factor of $4/3$ applies to certain alternative box or cell conventions, but it is not the standard axis-aligned bounding box of a regular hexagon. The important point is that using any oversized box systematically overestimates the total material volume and therefore the loop count.

2. Monte Carlo approximation of the Voronoi volumes

Exact three-dimensional clipped Voronoi cells can be expensive to construct. Monte Carlo integration estimates their volumes without explicitly building their faces.

Draw K points uniformly from the crystal:

$$\mathbf{y}_1, \dots, \mathbf{y}_K \sim \text{Uniform}(\Omega).$$

For each sample point, find its nearest node:

$$J(k) = \arg \min_j \|\mathbf{y}_k - \mathbf{x}_j\|.$$

Let

$$C_j = \sum_{k=1}^K \mathbf{1}[J(k) = j]$$

be the number of samples assigned to node j . The Monte Carlo estimate of the nodal volume fraction is

$$w_j = \frac{C_j}{K}.$$

Therefore,

$$\boxed{V_j = w_j V_{\Omega}}$$

and, because every sample belongs to exactly one node,

$$\sum_j C_j = K, \quad \sum_j w_j = 1, \quad \sum_j V_j = V_{\Omega}.$$

The counts follow a multinomial distribution:

$$(C_1, \dots, C_M) \sim \text{Multinomial}(K; w_1, \dots, w_M).$$

For an individual nodal fraction,

$$\text{Var}(\hat{w}_j) = \frac{w_j(1 - w_j)}{K},$$

so the standard Monte Carlo error decreases as $K^{-1/2}$.

A KD-tree or similar nearest-neighbor data structure reduces the assignment cost from approximately $O(KM)$ to roughly $O(K \log M)$.

3. Converting density into expected loop counts

Let $n(\mathbf{x})$ be the loop number density, with units

$$[n] = \text{loops/volume}.$$

If $n_j = n(\mathbf{x}_j)$, the expected number of loops associated with node j is approximated by

$$\lambda_j = n_j V_j.$$

This is a nodal quadrature approximation to

$$\lambda_j = \int_{V_j^\Omega} n(\mathbf{x}) dV \approx n_j V_j.$$

The total expected count is

$$\Lambda = N_{\text{expect}} = \sum_j \lambda_j = \sum_j n_j V_j.$$

In the continuum limit,

$$\sum_j n_j V_j \approx \int_{\Omega} n(\mathbf{x}) dV.$$

For uniform density $n_j = n_0$,

$$N_{\text{expect}} = n_0 \sum_j V_j = n_0 V_{\Omega}.$$

This is a useful conservation check.

4. Selecting the total number of loops

For a genuine inhomogeneous Poisson point process, the total count would be drawn as

$$N \sim \text{Poisson}(\Lambda),$$

with

$$\mathbb{E}[N] = \Lambda, \quad \text{Var}(N) = \Lambda.$$

Instead, the algorithm deliberately uses

$$N_{\text{draw}} = \text{round}(\Lambda).$$

This suppresses total-count fluctuations. The resulting error satisfies

$$|N_{\text{draw}} - N_{\text{expect}}| \leq \frac{1}{2}.$$

Strictly speaking, after replacing the Poisson draw by rounding, the complete process is no longer an unconditioned Poisson process. It is better described as a deterministic-count or approximately conditioned inhomogeneous spatial sampling process.

The spatial intensity is still derived from the Poisson measure; only the random fluctuation in the total count has been removed.

5. Assigning each loop to a home node

Normalize the nodal expected counts:

$$p_j = \frac{\lambda_j}{\Lambda} = \frac{n_j V_j}{\sum_k n_k V_k}.$$

The home node for each of the N_{draw} loops is sampled independently from the categorical distribution

$$J_\ell \sim \text{Categorical}(p_1, \dots, p_M), \quad \ell = 1, \dots, N_{\text{draw}}.$$

If N_j is the number of loops assigned to node j , then

$$(N_1, \dots, N_M) \sim \text{Multinomial}(N_{\text{draw}}; p_1, \dots, p_M).$$

Consequently,

$$\mathbb{E}[N_j] = N_{\text{draw}} p_j \approx \Lambda \frac{\lambda_j}{\Lambda} = \lambda_j,$$

so the expected nodal population remains approximately correct.

The nodal variance is

$$\text{Var}(N_j) = N_{\text{draw}} p_j (1 - p_j),$$

and the node counts are negatively correlated:

$$\text{Cov}(N_j, N_k) = -N_{\text{draw}} p_j p_k, \quad j \neq k,$$

because the total number is fixed.

6. Why both density and volume appear

Sampling based only on density,

$$p_j \propto n_j,$$

would incorrectly give every node equal geometric importance. This fails for irregular node spacing.

The correct weight is

$$p_j \propto n_j V_j.$$

Thus:

- n_j measures how many loops should occur per unit volume.
- V_j measures how much material the node represents.
- $n_j V_j$ measures how many loops that represented material should contain.

A node in a coarse region represents a larger volume and should therefore receive more loops than a node in a densely meshed region having the same physical density.

Algorithm

Input:

node coordinates x_j
nodal densities n_j
number of Monte Carlo samples K

1. Compute the convex hull Ω of the node cloud.
2. Compute its volume V_Ω .
3. Generate K points uniformly inside Ω .
4. For every point:
 - find the nearest node j
 - increment C_j
5. Compute:
 - $w_j = C_j / K$
 - $V_j = w_j V_\Omega$
6. Compute nodal intensities:
 - $\lambda_j = n_j V_j$
7. Compute:
 - $\Lambda = \sum_j \lambda_j$
 - $N_{\text{draw}} = \text{round}(\Lambda)$
8. Normalize:
 - $p_j = \lambda_j / \Lambda$
9. Draw N_{draw} home-node indices from $\text{Categorical}(p)$.
10. Generate loop properties and positions conditional on each home node

The essential mathematical chain is therefore

$$\text{geometry} \longrightarrow V_j \longrightarrow n_j V_j \longrightarrow p_j \longrightarrow \text{discrete loop population.}$$

Rounding makes dose-to-dose trends smoother, but spatial multinomial noise remains. If even that noise is undesirable in a dose sweep, one can use systematic or residual allocation instead of independent categorical sampling.